



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

MODULE WISE NOTES

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	MODULE WISE NOTES
Version	v1.0.0
Date	02 July 2026

What Was Built, Why, and What Was Learned — All 16 Modules

Module 1A — Project Initialisation

Overview

Set up the Vite + React development environment.

What Was Built

- Vite project scaffolded with React plugin
- `npm run dev` / `npm run build` / `npm run lint` scripts
- ESLint configuration
- Tailwind CSS v4 installed with `@tailwindcss/vite` plugin
- `.gitignore`, `.env`, `.env.example` created

Why Vite Over Create React App?

- **Build speed:** Vite serves modules using native ES modules — no bundling during dev
- **Build output:** Vite produces much smaller bundles than CRA
- **Modern:** CRA is deprecated; Vite is the industry standard

React Concepts Learned

- React project structure basics
- Module bundler concept (Vite)
- Environment variables (`VITE_` prefix)

Interview Questions

- Q: What is Vite? A: A build tool that uses native ES modules for dev and Rollup for production builds. Extremely fast compared to webpack-based tools.
 - Q: Why not Create React App? A: CRA is unmaintained and slow. Vite is the modern replacement.
-

Module 1B — Frontend Architecture

Overview

Set up the professional folder structure, routing skeleton, service layer, contexts, hooks, utilities, and configuration.

What Was Built

- Complete `src/` folder structure (components, pages, layouts, routes, services, hooks, contexts, utils, constants, config, styles)
- Route skeleton with React Router DOM
- Service files (authService, medicineService, pharmacyService, userService, inventoryService)
- Axios client with base URL
- Context files (AuthContext, ThemeContext, UserContext)

- Custom hooks (useDebounce, useLocalStorage, useOnlineStatus, useSessionGuard)
- Constants files (app.js, routes.js, navConfig.js)
- Utility files (formatters.js, validators.js, storage.js, authSchemas.js)

Design Decisions

- Service layer separation: API calls in services, not components
- Context for global state: avoids prop drilling
- Constants file for routes: no hardcoded strings

Backend Readiness

All service files pre-built — integrating the backend = uncommenting TODO lines

Interview Questions

- Q: Why separate service files? A: Separation of concerns. Components shouldn't know about API URLs. When the backend changes, only the service changes.
 - Q: What is prop drilling and how does Context solve it? A: Prop drilling = passing props through 5 layers. Context = any component reads directly from context.
-

Module 2 — Design System

Overview

Built the Tailwind v4 CSS-first design system — the visual foundation of the entire application.

What Was Built

- `src/index.css` with complete `@theme {}` token definitions
- Color palette (primary blue, teal, success, warning, danger, info)
- Typography scale (Inter font, xs to display sizes)
- Spacing tokens
- Shadow tokens
- Border radius tokens
- Z-index system
- Animation keyframes (`page-enter`)
- `src/styles/tokens.js` — JS mirror of design tokens
- `src/styles/theme.js` — light/dark theme configuration

Why CSS-First Design Tokens?

Tailwind v4's `@theme {}` creates CSS custom properties AND Tailwind utility classes from one definition. Change `--color-primary-600` once → updates everywhere.

Interview Questions

- Q: What is a design system? A: A set of visual rules (colours, fonts, spacing) that ensure consistency across all pages.
 - Q: Why use design tokens? A: To avoid hardcoded hex values in 200 components. Change the brand colour in one place, updates everywhere.
-

Module 3A — Core UI Components

Overview

Built the atomic UI components — the smallest reusable building blocks.

What Was Built

- **Button** — 5 variants, 3 sizes, loading state, icons
- **Input** — label, error, icons, helper text
- **PasswordInput** — show/hide toggle
- **PasswordStrength** — strength indicator bar
- **Textarea** — multi-line input
- **Select** — styled dropdown
- **Checkbox** — styled with label
- **RadioGroup** — option group
- **Toggle** — on/off switch
- **Badge** — 8 variants, dot, icon
- **Avatar** — image with initials fallback, status dot
- **Divider** — separator line
- **OtpInput** — 6-box auto-advance OTP
- **FormField** — wrapper with label + error

Design Decisions

- All components are prop-driven — no hardcoded content
- All interactive elements have **focus-visible** styles for accessibility
- All error states use ARIA **aria-describedby**

React Concepts Learned

- Controlled vs uncontrolled inputs
 - React Hook Form integration pattern
 - Forwarded refs for input components
-

Module 3B — Advanced Components

Overview

Built all remaining reusable components — navigation, cards, dialogs, feedback.

What Was Built

- **Navigation:** NavBar, Footer, Sidebar, TopBar
- **Cards:** MedicineCard, PharmacyCard, NotificationCard, InfoCard, SearchResultCard
- **Common:** Breadcrumb, SearchBar, Pagination, AppErrorBoundary
- **Dialogs:** Modal, ConfirmDialog
- **Feedback:** Spinner, Skeleton, EmptyState, ErrorState
- **Layout:** Container, PageHeader, SectionHeader

Key Decisions

- **Barrel exports** (`index.js` per folder): `import { Button, Badge } from '../ui'` instead of individual imports
- **Compound components** (SearchResultCard has sub-components SmartBadges, CardActions, MedicineImagePlaceholder)

Module 4 — Navigation & Layout System

Overview

Integrated navigation components into five layouts. Each layout wraps its page group.

What Was Built

- `MainLayout` — Navbar + Footer + Outlet
- `AuthLayout` — Centered card layout
- `UserLayout` — Sidebar + TopBar + Outlet with `page-enter` animation
- `PharmacyLayout` — Same as UserLayout with pharmacy nav
- `AdminLayout` — Same as UserLayout with admin nav
- NavConfig (`navConfig.js`) drives Sidebar nav items per role

React Concepts Learned

- `<Outlet />` pattern for nested routes
- Layout persistence between page navigations
- Role-based navigation config

Module 5 — Authentication Pages

Overview

Built all five authentication pages with full form validation.

What Was Built

- `LoginPage` — email + password + rememberMe
- `RegisterPage` — full registration with password strength
- `VerifyOtpPage` — 6-box OTP + countdown timer + resend
- `ForgotPasswordPage` — email submission
- `ResetPasswordPage` — new password + confirm

Form Validation Architecture

- Zod schemas in `utils/authSchemas.js` — separate from components
- `@hookform/resolvers` connects Zod to React Hook Form
- Pattern: `useForm({ resolver: zodResolver(loginSchema) })`

AuthContext Integration

- All auth actions (`login`, `register`, `verifyOtp`) in `AuthContext.jsx`
- Mock responses for frontend-only development
- TODO comments show exactly where to swap in real API calls

Interview Questions

- Q: Why React Hook Form? A: Uncontrolled inputs avoid re-render on every keystroke. Built-in error handling. Much less boilerplate than useState per field.
 - Q: Why Zod for validation? A: Type-safe schema validation. Reusable schemas shared between frontend validation and can be shared with backend TypeScript.
-

Module 6 — Home Page

Overview

Built the public landing page with 9 sections showcasing the project.

What Was Built

- [HomePage](#) composed of 9 lazy-loaded section components
- HeroSection, AboutSection, JanAushadhiSection, FeaturesSection, HowItWorksSection, BenefitsSection, StatsSection, FaqSection, CtaSection

Performance Decision

HomePage sections are individually code-split (separate JS chunks). Each section imports and renders independently. Users see content as sections load progressively.

Module 7A — Medicine Search Page

Overview

Built the rich medicine search interface.

What Was Built

- Full-width search bar with [useDebounce](#)
- SearchSuggestions autocomplete panel
- RecentSearches (localStorage via [useLocalStorage](#))
- PopularMedicines grid
- MedicineCategories browse grid
- SearchFilters panel (type, availability, price, manufacturer)
- SearchLoadingState, SearchEmptyState, SearchErrorState

Hook Usage

```
const debouncedQuery = useDebounce(searchQuery, 400)
// Prevents API call on every keystroke
// Only searches after 400ms of no typing
```

Module 7B — Search Results + Medicine Details

Overview

Two complex pages — the search results grid and the full medicine detail view.

Search Results Built

- SearchSummarySection (query + results count)
- ResultsToolbar (sort, grid/list toggle, compare mode)
- ResultsGrid (SearchResultCard grid)
- ActiveFilters chips
- CompareBar (sticky bottom — selected medicines)
- Pagination

Medicine Details Built

- MedicineHeader (name, badges, price comparison)
 - ActionPanel (View Generic, Save, Share, Nearby buttons)
 - MedicineInfoTabs (Overview | Composition | Usage | Side Effects | Storage)
 - GenericRecommendationSection (teaser)
 - NearbyPharmaciesTeaser
 - PriceComparison table
 - RelatedMedicines
 - HealthcareDisclaimer
-

Module 8 — Generic Recommendation Page

Overview

The core feature page — recommends Jan Aushadhi generic alternatives.

What Was Built

15-section page including:

- BrandedMedicineSummary
- WhyGenericsSection (educational)
- GenericAlternativesList with price comparison
- CostSavingsCalculator (interactive)
- EvidenceSection
- HowToSwitchGuide
- HealthAwarenessSection
- Disclaimer sections

Healthcare Importance

This is the central value proposition of the system — showing patients how much they can save by switching to generics.

Module 9 — Nearby Pharmacies + Leaflet Map

Overview

Interactive map showing nearby Jan Aushadhi pharmacies.

What Was Built

- React Leaflet `<MapContainer>` with OpenStreetMap tiles
- Custom pharmacy marker pins
- Marker click → Popup with pharmacy info
- PharmacyListPanel (sortable by distance)
- FilterPanel (open now, has stock)
- Placeholder coordinates (Mumbai — will be replaced by Geolocation API + backend)

Key Package

`react-leaflet` wraps the `leaflet` map library for React. `<MapContainer>`, `<TileLayer>`, `<Marker>`, `<Popup>` are the key components.

Module 10 — User Dashboard

Overview

The patient's home screen after login.

What Was Built

10 sections including:

- WelcomeBanner (personalised with user name)
- QuickActions (shortcuts to main features)
- RecentSearches
- SavedMedicines
- RecentNotifications
- HealthTips
- JanAushadhiAwareness
- DashboardStats
- ProfileCompletion
- NearbyPharmacyTeaser

Reuse Achievement

Reused: InfoCard, NotificationCard, MedicineCard, Avatar, Badge, Toggle, Button — zero new UI components needed.

Module 11 — Pharmacy Dashboard

Overview

Built the pharmacist's tools — dashboard, inventory management, add/edit stock.

What Was Built

- `PharmacyDashboard` with stats and quick actions

- [InventoryPage](#) — searchable, sortable, paginated medicine table
- [MedicineFormPage](#) — shared form for Add and Edit (mode determined by URL `:id`)
- Mock data in [pages/pharmacy/data/inventoryData.js](#)

Same Form for Add and Edit Pattern

```
function MedicineFormPage() {
  const { id } = useParams()
  const isEditMode = Boolean(id)
  // Same form, different behavior based on isEditMode
}
```

Module 12 — Admin Portal

Overview

Built the complete 11-page admin portal.

What Was Built

- AdminDashboard, AdminUsers, AdminPharmacies, AdminMedicines, AdminGenericMapping
- AdminAnalytics, AdminReports, AdminNotifications, AdminActivity, AdminRoles, AdminSettings
- Shared [AdminTable](#) component (reused across all table pages)
- Mock data in [pages/admin/data/adminData.js](#)

Architecture Achievement

352 modules in the full build — kept manageable by the consistent AdminLayout + AdminTable pattern.

Module 13 — Frontend Integration & Error Handling

Overview

Connected all pages into a cohesive application with error handling.

What Was Built

- [NotificationsPage](#) (full implementation)
- [ProfilePage](#)
- [SessionExpiredPage](#)
- [ServerErrorPage](#)
- [OfflinePage](#)
- [AppErrorBoundary](#) (class component)
- [useOnlineStatus](#) hook integrated into App.jsx

Build Result

- Lazy loading: main bundle 464KB (down from 963KB without code splitting)
 - 43 code-split chunks
-

Module 14 — Production Readiness

Overview

Code cleanup and polish pass.

What Was Done

- Removed all unused imports
 - Added `no-unused-vars` ESLint rule for `.js` files
 - Fixed `UnauthorizedPage` icon import
 - Added `page-enter` CSS animation to all 4 layout `<main>` elements
-

Module 15 — Release Candidate v1.0

Overview

Final release preparation.

What Was Done

- Version bumped to `1.0.0` in `package.json`
 - Comprehensive `.env.example` documented
 - `axiosClient.js` upgraded with `/session-expired` redirect on 401
 - `README.md` — complete project documentation
-

Module 16 — Senior Architecture Review

Overview

Final code quality pass by a senior architect perspective.

What Was Fixed

1. `ROUTES.SESSION_EXPIRED` constant added to `routes.js`
2. `PHARMACY.INVENTORY_ADD` and `PHARMACY.INVENTORY_EDIT` added to `routes.js`
3. `axiosClient.js` — replaced hardcoded `/session-expired` string with `ROUTES.SESSION_EXPIRED`
4. `App.jsx` — removed duplicate loading text
5. `SearchResultCard.jsx` — added `onShare` to props signature + Web Share API integration
6. `UserContext.jsx` — improved TODO comment accuracy
7. `index.html` — added `robots` meta tag + `<noscript>` fallback

Final Build Verification

- ■ 0 lint errors
- ■ 0 build errors
- ■ 360 modules transformed
- ■ 43 code-split chunks
- ■ Main bundle: 464KB (gzip: 138KB)